Invobook: Next-Generation Invoice & Inventory Management System

BCA 8th Semester Project Documentation

---

1. Introduction

Invobook is a comprehensive, cloud-based Invoice and Inventory Management System designed to streamline financial tracking for freelancers, small businesses, and enterprises. Built with a modern tech stack and a premium, Vercel-inspired UI/UX, Invobook simplifies the entire billing lifecycle—from client management and inventory tracking to invoice generation, payment tracking, and analytics.

---

2. Problem Statement & Solution

The Problem

Traditional billing systems and manual ledger entries are prone to human error, difficult to scale, and lack real-time insights. Small businesses often struggle with disjointed tools—using one software for inventory, another for invoicing, and yet another for tracking payments. This leads to data silos, unpaid invoices slipping through the cracks, and a poor experience for both the business owner and their clients.

The Solution (Invobook)

Invobook provides a unified, single-pane-of-glass dashboard that tightly integrates **Invoicing** and **Inventory Management**.

- **Automation**: Instantly calculates taxes, discounts (fixed/percentage), shipping, and partial payments.
- **Unified Ecosystem**: Products added to an invoice are automatically deducted from the inventory system in real-time.
- **Instant Sharing**: Invoices can be shared instantly via WhatsApp, Email, or downloaded as pixel-perfect PDFs and Images.
- **Premium UX**: A distraction-free, highly responsive interface built with accessibility in mind, reducing the learning curve to zero.

---

3. Technologies & Services Used

Invobook leverages a cutting-edge full-stack architecture optimized for serverless deployment on Vercel.

Frontend:

- **Next.js 15 (Pages Router)**: Provides robust Server-Side Rendering (SSR) and API routes.
- **React 19**: Drives the dynamic, component-based user interface.
- **Tailwind CSS v4**: Handles utility-first, highly responsive, and dark-mode-ready styling.
- **Radix UI**: Provides headless, fully accessible UI components (Dropdowns, Dialogs, Popovers) for the Vercel-style design system.
- **Lucide React**: Supplies clean, modern SVG iconography.

Backend & Database:

- **Next.js API Routes**: Acts as the serverless backend infrastructure.
- **Prisma ORM**: Provides type-safe database querying and schema management.
- **PostgreSQL**: The robust, scalable relational database used for all data persistence.
- **JWT & Bcrypt**: Handles secure authentication, session management, and password hashing.

External Services & Micro-Libraries:

- **Puppeteer-Core & Sparticuz/Chromium**: Powers the serverless PDF and PNG image generation engine for invoices, bypassing standard HTML-to-PDF limitations.
- **Vercel Blob (Optional Integration)**: For secure cloud storage of business logos and payment proof receipts.

---

4. Module Breakdown & Core Features

4.1. Authentication & User Management

- Secure JWT-based login/registration system.
- Per-user data isolation (multi-tenancy approach).
- Customizable business profiles (Logo, Name, Address, Tax ID).

4.2. Invoice Management Engine

- **Dynamic Creation**: Build invoices with unlimited items. Supports custom rates, variable quantities, and descriptions.
- **Advanced Calculations**: Real-time computation of Subtotal, Tax Rates, Shipping Costs, and Discounts (Percentage or Flat).
- **Payment Tracking**: Granular tracking of invoice status (`DRAFT`, `PENDING`, `PARTIALLY_PAID`, `PAID`, `OVERDUE`).
- **Export & Share**: Server-side rendering engine converts HTML invoices into highly accurate PDFs and PNGs for immediate download or sharing via WhatsApp.

4.3. Inventory Management (New Addition)

- **Stock Tracking**: Users can maintain a catalog of items with SKUs, units, and rates.
- **Smart Validation**: When enabled, the invoice creator forces users to select items directly from inventory, preventing the addition of "random" untracked items.
- **Low Stock Alerts**: Configurable thresholds to warn the user when inventory runs low.

4.4. Client Management

- Centralized Rolodex for managing client details (Email, Phone, Address, Tax ID).
- One-click association of clients to invoices.

4.5. Dashboard & Analytics

- Real-time statistics cards displaying Total Revenue, Outstanding Balances, and Total Invoices.
- Recent activity feeds and quick-action menus.

---

5. UI/UX Improvements & Design Philosophy

Invobook stands out due to its relentless focus on a premium, "Pro-tier" aesthetic inspired by the Vercel design system:

- **Unified Navigation**: Replaced legacy, cluttered headers with a sleek `SubNav` architecture and a unified `InvoiceActionMenu` (Three-dot dropdown) across all table and grid views.
- **Radix UI Integration**: Migrated complex modal and dropdown logic to Radix UI for smooth animations (slide-up/slide-down), keyboard navigation, and reliable out-of-bounds click detection.
- **Dynamic Views**: Users can seamlessly toggle between 'List View' (Table) and 'Grid View' (Cards) based on their preference, with state consistency maintained.
- **Micro-Interactions**: Features subtle hover effects, active state styling, and glass-morphism techniques to make the interface feel alive and highly responsive.
- **Accessibility & Security**: Implemented strict `upgrade-insecure-requests` CSP headers to prevent mixed-content warnings on strict enterprise networks.

---

6. Database Schema Breakdown (Prisma)

The relational schema is highly normalized to ensure data integrity:

1. **User**: The root entity. Handles authentication and global settings.
2. **Business**: (1:1 with User) Stores company branding and global configuration.
3. **InvoiceSetting**: (1:1 with Business) Stores defaults like Due Days, Tax Rates, and Payment Instructions.
4. **Client**: (1:M with User) Stores customer profiles.
5. **InventoryItem**: (1:M with User) Stores trackable products/services.
6. **Invoice**: (1:M with User, 1:M with Client) The core transactional entity. Tracks amounts, discounts, and payment statuses.
7. **InvoiceItem**: (1:M with Invoice) Represents individual line items on a specific invoice.
8. **Payment**: (1:M with Invoice) Tracks partial and full payment ledgers against an invoice.
9. **Notification**: (1:M with User) Real-time event logging system.

---

7. Standout Algorithms & Technical Highlights

To achieve enterprise-grade reliability and automation, Invobook implements several advanced algorithms that set it apart from basic CRUD applications.

A. Automated Inventory-Ledger Synchronization Algorithm

Managing stock levels concurrently across multiple invoices requires strict transaction control to prevent race conditions (overselling).
Algorithm Flow:

1. **Input:** An array of `InvoiceItems` with `id`, `sku`, and `quantity`.

2. **Locking & Validation:** The system initiates a Prisma Database Transaction (`$transaction`). It queries the `InventoryItem` table to fetch current stock levels.
3. **Deduction & Rollback Check:** For each item, `new_quantity = current_quantity - requested_quantity`. If `new_quantity < 0`, the entire transaction is automatically rolled back, throwing an "Insufficient Stock" error.
4. **Commit & Trigger:** If all constraints pass, the database commits the new quantities.
5. **Async Notification:** A background worker compares the `new_quantity` against the user's predefined `lowStock` threshold. If breached, it inserts a `Notification` payload to alert the user in real-time.

B. The Serverless HTML-to-Stream Rendering Algorithm (PDF Export)

Traditional client-side PDF generation (like `jspdf`) breaks modern CSS grid/flexbox layouts. Invobook solves this using an isolated, serverless browser engine (`@sparticuz/chromium`):
Algorithm Flow:

1. Client requests an export for `invoiceId=XYZ`.
2. A Vercel Serverless Function spins up a headless Chromium instance.
3. The instance navigates to a hidden, authenticated `/preview` route containing the exact HTML/CSS DOM of the invoice.
4. Chromium waits for network idle (fonts and CSS loaded), then executes a `page.pdf()` or `page.screenshot()` command.
5. The resulting binary Buffer is streamed directly back to the client via HTTP response headers (`Content-Disposition: attachment`), bypassing the slow and expensive filesystem completely.

C. Dynamic Cascade Calculation Algorithm (Financials)

Handling financial math in JavaScript is notoriously dangerous due to floating-point precision errors (e.g., `0.1 + 0.2 = 0.30000000000000004`).
Algorithm Flow:

1. All monetary values are strictly cast to Prisma `Decimal` types.
2. **Row-Level Loop:** The algorithm maps over all invoice items, calculating `row_amount = item.quantity * item.rate`.
3. **Subtotal Aggregation:** `subtotal = sum(row_amounts)`.
4. **Discount Cascade:** If `DiscountType === PERCENTAGE`, then `discount = subtotal * (discountValue / 100)`. Else, `discount = discountValue`.
5. **Tax & Total:** `taxAmount = (subtotal - discount) * (taxRate / 100)`. Finally, `Total = (subtotal - discount) + taxAmount + shippingCost`.
6. This algorithm guarantees exact penny-precision regardless of the complexity of the invoice.

D. Real-Time Partial Payment Reconciliation Algorithm

When clients make payments (via bank transfer or QR code), the invoice balance must automatically reflect this.
Algorithm Flow:

1. A new `Payment` object is created and linked to the `Invoice`.
2. The system triggers an aggregation: `total_paid = sum(payments.amount) where status == "verified"`.
3. `balanceDue = Invoice.total - total_paid`.
4. **Status Mutation:** If `balanceDue == 0`, `status = PAID`. If `0 < balanceDue < Invoice.total`, `status = PARTIALLY_PAID`. Otherwise, `PENDING`.

---

8. Conclusion

Invobook successfully bridges the gap between enterprise-grade financial software and consumer-friendly design. By utilizing a modern Next.js serverless architecture and a highly polished UI/UX, the platform provides a robust, scalable, and extremely fast solution for modern billing and inventory needs. It stands as a comprehensive, real-world application suitable for rigorous academic evaluation and commercial deployment.